

# AI Telehealth Platform — Architecture Diagram (Detailed Reference)

**System Design Series · Document 2 of 4** (ERD → **Architecture** → Sequence Diagrams → API Contract) **Stack:** ASP.NET Core 10 (Clean Architecture, CQRS/MediatR) + Flutter (Clean Architecture, BLoC)

Figure 1: Architecture Diagram

---

## 1. ERD vs. Architecture Diagram — What’s the Difference?

These two diagrams answer two completely different questions:

- **ERD (Entity Relationship Diagram):** answers “*how is the data stored, and how are the tables related to each other?*” — it’s about persistence.
- **Architecture Diagram:** answers “*what does the whole system consist of, and how does each part talk to the others?*” — it’s about components and communication.

In short: the ERD tells you the shape of the data; the architecture diagram tells you the shape of the system. Both are needed, and they should stay consistent with each other — for example, the Architecture Diagram says “there is a relational database,” and the ERD says “here is exactly what that database looks like.”

## 2. The Big Picture

The diagram above is the final, agreed architecture for v1. Before breaking it down piece by piece, here is the reasoning behind why each component exists — understanding the *why* matters far more than memorizing the picture.

---

## 3. Flutter Applications (Client Layer)

The project has one Flutter app with two Flavors — a **Patient flavor** and a **Consultant flavor** — both built with Clean Architecture and BLoC. As a client, Flutter’s responsibilities are:

- UI and user interaction
- Authentication flows (login, token storage, refresh)
- Calling the backend API
- Displaying appointments, chat, and notifications
- Initiating WebRTC calls
- Payment UI (collecting card details via Stripe’s client SDK, never touching the secret key)

**The one rule that matters most here: Flutter must never contain sensitive business rules.** The client can be wrong, outdated, or tampered with — so it should never be trusted to make decisions that affect data integrity or security.

**Wrong (Flutter decides):**

Flutter sees "slot looks free" -> assumes it can book it directly

### Correct (Backend decides):

```
Flutter
|
POST /appointments
|
Backend: check availability, check conflicts, create appointment
|
Response: confirmed or rejected
```

This directly matches the booking concurrency rules already locked in the Requirements Document and ERD — the backend, not the client, is the single source of truth for whether a slot is actually available.

**Why one app with Flavors instead of two separate apps?** Both flavors share the same Clean Architecture codebase (domain, data, BLoC, SignalR client, API clients, models). The only difference is the entry point (`main_patient.dart` vs `main_consultant.dart`) and the presentation layer (screens, navigation). This eliminates code duplication, simplifies maintenance, and speeds up the MVP. Each flavor is built as a separate binary with its own `applicationId` and icon, so the stores treat them as distinct apps even though they share a single codebase.

## 4. ASP.NET Core 10 API (Application Layer)

This is the heart of the system. Every client — Patient flavor, Consultant flavor — talks only to this API. It owns:

- Authentication & Authorization
- Appointments & Scheduling
- Payments
- Consultations
- Chat (persistence side)
- Notifications
- AI orchestration

A single giant controller handling all of this would be unmanageable. Instead, the API is organized into feature modules, and each module internally follows Clean Architecture (Data / Application / Domain / Presentation), consistent with how the Restaurants API and Safaria backend were already structured:

```
ASP.NET Core
├─ Auth
├─ Users
├─ Patients
├─ Consultants
├─ Appointments
├─ Consultations
├─ Payments
├─ Chat
├─ Notifications
└─ AI
```

This is what's meant by a **Modular Monolith**: one deployable application, but internally organized as clearly separated modules — giving most of the maintainability benefits of microservices without the

operational overhead of running and coordinating several separate services. That overhead is explicitly not worth taking on for this project (see Section 15).

---

## 5. SQL Server (Primary Database)

The system's single source of truth for durable data: Users, PatientProfiles, ConsultantProfiles, Appointments, Consultations, Payments, ChatMessages, AISummaries, AIJobs, Notifications, RefreshTokens. This maps directly onto the ERD built earlier — the relationship between the two documents is simple:

```
Architecture Diagram says: "there is a relational database."  
ERD says:                  "here is exactly what that database looks like."
```

---

## 6. Redis — Optional in v1

**Redis is not a replacement for SQL Server.** It serves a different purpose: fast, ephemeral data. In this project it would be used for two things:

- **Caching** — e.g. consultant availability lookups, so not every request hits SQL Server directly.
- **SignalR backplane** — needed only if the API scales to more than one instance, so real-time messages reach users connected to a different server instance.

```
      ┌─ API #1  
Flutter ───┤─ API #2  
      └─ API #3
```

Since v1 runs on a single instance, Redis is deliberately deferred rather than added “because production systems have it.” This is a documented decision, not an oversight — see the Requirements Document, Section 8 (“Decisions Made”) and Section 6 (“Later — v2+”).

---

## 7. Background Workers

Not everything the API does should happen synchronously inside an HTTP request. The clearest example in this system is generating an AI consultation summary — a slow, unreliable, external-dependency call that should never block a client waiting on a response.

**What NOT to do:**

```
POST /consultations/{id}/complete  
|  
Call AI -> wait 10 seconds -> generate summary -> return response    (BAD design)
```

**What the system actually does instead:**

```
API  
|  
Create AIJob (Status = Pending)  
|  
Return success immediately  
|
```

```
Hosted BackgroundService picks up the job asynchronously
|
Process, retry on failure, persist result, notify
```

---

## 8. AI Job — Why the Pipeline is Designed This Way

The ERD already has an **AIJob** entity, which means the system was designed from the start to treat AI processing as its own asynchronous unit of work, not something bolted onto a controller:

```
Consultation Completed
    ↓
  AIJob (Status = Pending)
    ↓
Hosted BackgroundService
    ↓
  IAIService
    ↓
  AI Provider (OpenAI)
    ↓
Structured Output (validated)
    ↓
AISummary persisted
```

This is a deliberately better design than a naive **Controller** → **OpenAI** call, because it survives provider slowness, transient failures, and — thanks to the `ProcessingStartedAt` stuck-job recovery logic already defined in the Requirements Document — even a server restart mid-job.

---

## 9. AI Service — The Abstraction Layer

The Application layer never talks to OpenAI directly. It talks to an interface:

```
IAIService
|
├─ OpenAIService          (v1 implementation)
└─ <FutureProviderService> (swap-in later, e.g. Azure OpenAI)
```

The rest of the application only ever calls something like `GenerateSummary(...)` — it has no idea whether that's backed by OpenAI, Azure OpenAI, or anything else. This is exactly the **IAIService** → `Microsoft.Extensions.AI` → `OpenAI` chain already locked in as a v1 decision, and it's what gives the system the flexibility to change AI providers later without touching business logic.

---

## 10. Stripe (Payments)

```
Flutter
|
ASP.NET Core
```

```
|
Stripe
```

Flutter never handles the Stripe secret key. The backend creates the `PaymentIntent` and returns only what the client needs (the client secret) to complete payment on its side. The webhook is what closes the loop:

```
Stripe
|
Webhook
|
ASP.NET Core
|
Update Payment status
```

This is the same rule already established in the Requirements Document: **the system never trusts the Flutter client's claim that a payment succeeded** — only the Stripe webhook is treated as the source of truth for payment confirmation.

## 11. WebRTC — A Fundamentally Different Kind of Connection

This part deserves special attention because it breaks the usual client-server-client pattern. The actual audio/video media must **not** flow through the ASP.NET Core API:

**Wrong (inefficient — API becomes a media relay):**

```
Flutter -> ASP.NET Core -> Flutter
```

**Correct (peer-to-peer media, backend only brokers the handshake):**

```
Patient device (Patient flavor)

    WebRTC

Consultant device (Consultant flavor)
```

The backend's role in a call is limited to:

- Signaling (exchanging Offer / Answer / ICE Candidates)
- Authentication & Authorization (only appointment participants may join)
- Call lifecycle events (started, ended, no-show, etc.)

```
Flutter
|
SignalR / Signaling
|
ASP.NET Core
|
Exchange SDP / ICE
|
Direct WebRTC connection (STUN, or TURN as fallback)
```

---

## 12. SignalR — Real-Time Delivery

SignalR is the backbone for both chat and WebRTC signaling. For a chat message specifically:

```
Patient
|
SignalR
|
ASP.NET Core
|
Consultant
```

When the patient sends “*Hello Doctor*”, the server does two things in parallel:

```
Save message -> SQL Server
|
Broadcast in real time -> Consultant
```

**SQL Server is the persistence layer; SignalR is the real-time delivery layer.** This distinction matters — losing the SignalR connection should never mean losing the message, because the database write does not depend on the delivery succeeding.

---

## 13. Notifications

There are two notification channels, serving different purposes:

- **In-app (v1):** SignalR → Flutter, delivered while the app is open.
- **Push (v2):** Firebase Cloud Messaging, delivered even when the app is closed.

```
Appointment confirmed
↓
Notification record (persisted)
↓
Push notification (v2, via FCM)
```

The `Notifications` table is the persistent history of what happened; the delivery channel (SignalR now, FCM later) is just how the person finds out about it. Keeping these separate means adding push notifications later doesn’t require changing how notifications are generated or stored.

---

## 14. What Actually Belongs in an Architecture Diagram?

An architecture diagram answers one question: “**what is the system made of?**” — not “what are all its tables and classes?” That’s a different, more detailed diagram (ERD / Component / Class design).

---

|                                 |                                                   |
|---------------------------------|---------------------------------------------------|
| Belongs in Architecture Diagram | Belongs in ERD / Component / Class Design instead |
| Client                          | Appointment                                       |

---

Table 1 – continued

| Belongs in Architecture Diagram | Belongs in ERD / Component / Class Design instead |
|---------------------------------|---------------------------------------------------|
| API                             | PatientProfile                                    |
| Database                        | AIJob                                             |
| Cache                           | Payment                                           |
| Background Worker               | (any individual table or class)                   |
| AI Provider                     |                                                   |
| Payment Provider                |                                                   |
| Realtime layer                  |                                                   |

## 15. What This Diagram Deliberately Leaves Out

None of the following are recommended right now — not because they’re bad technologies, but because adding them without a real, present need is premature complexity for a learning project:

- Kubernetes
- Kafka
- Microservices
- RabbitMQ / message brokers
- Multiple databases
- API Gateway
- Load Balancer
- Redis Cluster

The goal for v1 is a **Production-Ready Modular Monolith** — a design that’s not so simple it skips real system-design learning, and not so over-engineered that time gets spent on infrastructure instead of backend architecture:

```
Flutter (Patient flavor / Consultant flavor)
  |
ASP.NET Core Modular Monolith
  |
SQL Server
  |
Background Worker

+ SignalR, Stripe, WebRTC, AI Provider (as needed)
+ Redis (only once there's a real, present need for it)
```

## 16. Authentication & Authorization — Token Flow

The Requirements Document already says Flutter handles login/token storage/refresh, and the API handles authentication/authorization — but it’s worth making the actual token flow explicit, especially since `RefreshTokens` already exists in the ERD:

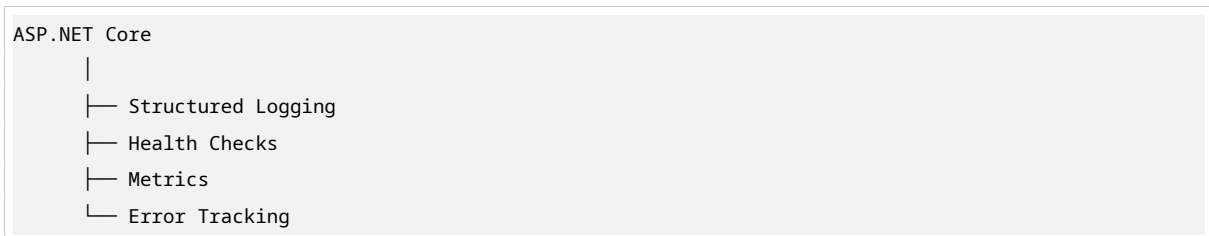


No external Auth Server is needed for this. ASP.NET Core's own Identity + JWT handling is sufficient for v1 — this is purely about making an already-decided flow visible in the diagram, not a new component.

---

## 17. Observability (v1 Baseline)

If the goal is learning patterns that transfer to real production work, this is arguably the most valuable addition — and the easiest to skip by accident:



This does **not** mean standing up Grafana + Prometheus + ELK in v1. A realistic v1 baseline is:

- ASP.NET Core's built-in structured logging
- Global exception handling middleware
- Health check endpoints (`/health`)
- A correlation ID per request (useful for tracing a request across the API → BackgroundService → AI pipeline)
- A hosted error-tracking service (e.g. Sentry or Application Insights) for anything beyond local logs

The concept mattering more than the specific tool is the point — the system should never be a black box once something goes wrong.

---

## 18. Notification Channel Abstraction (Concept Now, Implementation Later)

Notifications in v1 stay exactly as already decided: `Notification` → `SignalR` → `Flutter`, with FCM push deferred to v2. That doesn't change.

What's worth adding at the *design* level (not the implementation level) is the shape the abstraction should take once more channels are added:

```
INotificationService
|
├─ InApp    (v1 — implemented)
├─ Push     (v2)
├─ Email    (v2+)
└─ SMS      (v2+)
```

Nothing here needs building now — the value is simply designing the `Notifications` table and the calling code so that adding a channel later means adding an implementation of the interface, not restructuring how notifications are triggered.

---

## 19. Storage (Object/File Storage) — Explicitly Deferred

The Requirements Document already defers `MedicalRecords` and `Attachments` to v2, so file/object storage should **not** be added to the v1 architecture just for completeness. Adding infrastructure ahead of an actual requirement is exactly the kind of premature complexity Section 15 already warns against.

For reference, once v2 needs it:

```
ASP.NET Core
|
▼
Object Storage
|
├─ Medical Documents
├─ Attachments
└─ Images
```

This is documented here only so the shape is known in advance — it is **not** part of the v1 diagram.

---

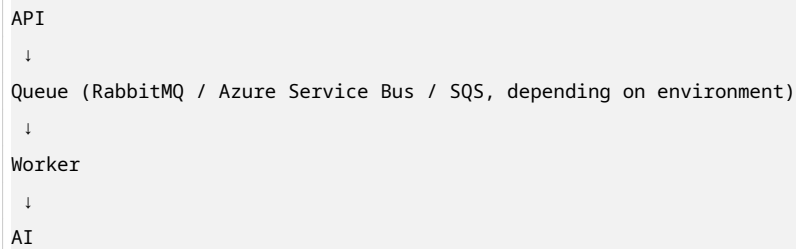
## 20. Message Queue — A Future Evolution Path, Not a v1 Component

The current pipeline —

```
ASP.NET Core
  ↓
Hosted BackgroundService
  ↓
AIJob
```

— is the right fit for v1, and matches the existing decision to avoid Hangfire or a message broker for now (Requirements Document, Section 8). If AI workload ever grows large enough to need real queuing/

distribution:



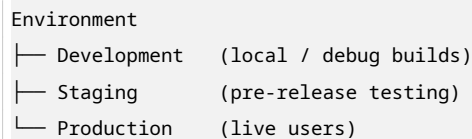
This is a **later evolution path**, not a v1 component — recorded here so the migration direction is clear if it's ever needed, without pulling the infrastructure in early.

## 21. Deployment Architecture (Separate from Logical Architecture)

Everything above is the **Logical Architecture** — what the system is made of and how components communicate. A full system-design pass also benefits from a small, separate **Deployment Architecture** diagram — how those same components actually get deployed and run:

Figure 2: Deployment Architecture

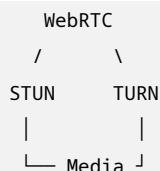
The same modular monolith is deployed as a single instance in v1 across three environments:



Keeping this diagram separate from the logical one matters: the *logical* architecture (modules, data flow, abstractions) stays the same across environments — only *how* and *where* it's deployed changes. Conflating the two makes both diagrams harder to read.

## 22. WebRTC — STUN vs. TURN, Made Explicit

The original diagram already showed STUN/TURN correctly, but it's worth stating the distinction plainly since it's easy to gloss over:



- **STUN** — used first, attempts to establish a **direct peer-to-peer** connection by discovering each client's public-facing address (NAT traversal).
- **TURN** — only used as a **fallback** when a direct P2P connection isn't possible (e.g. restrictive NAT/firewall on one side); traffic is relayed through the TURN server instead of going directly between clients.

In both cases, ASP.NET Core never carries the audio/video itself — its only role stays signaling (exchanging the offer/answer/ICE candidates so the two clients can find each other), consistent with Section 11.

---

## 23. Updated Assessment

Rating the architecture as it now stands, purely from a “learning production-grade system design” lens: **8.5–9/10.**

The gap was never a missing *technology* — it was under-specifying a few *concerns*:

- 🗝️ Authentication / Authorization flow — now explicit (Section 16)
- 📊 Observability — now included as a v1 baseline (Section 17)
- 📧 Notification channel abstraction — documented as a future-ready shape (Section 18)
- 💾 Storage — explicitly deferred, not added prematurely (Section 19)
- 🚀 Queue — documented as a future evolution path, not a v1 component (Section 20)
- 🌐 Deployment architecture — now a separate diagram (Section 21)
- STUN vs. TURN — made explicit (Section 22)

The things still deliberately excluded remain the same, and for the same reason: **Kubernetes, Kafka, RabbitMQ, Microservices, API Gateway, Load Balancer, Redis Cluster.** The project’s own decision — a v1 Production-Ready Modular Monolith — hasn’t changed; these additions sharpen it rather than complicate it.

---

**Next in the System Design series:** Sequence Diagrams for the three most complex flows — Booking → Payment → Confirmation, WebRTC call establishment, and the async AI Summary pipeline — followed by the full API Contract.